Pruning of Deep Neural Networks for Real-Time Execution on Edge TPU

Mouad Zouhdi Jiahe Guo Rafik Hammadi Binqi Sun
Philippe Leleux Tomasz Kłoda Marco Caccamo

LAAS-CNRS, Université de Toulouse, INSA Toulouse,
Sorbonne Université, EURECOM, France
Technical University of Munich, Germany

Euromicro Conference on Digital System Design (DSD) 2026
Kraków, Poland

Edge TPU

Coral USB Accelerator

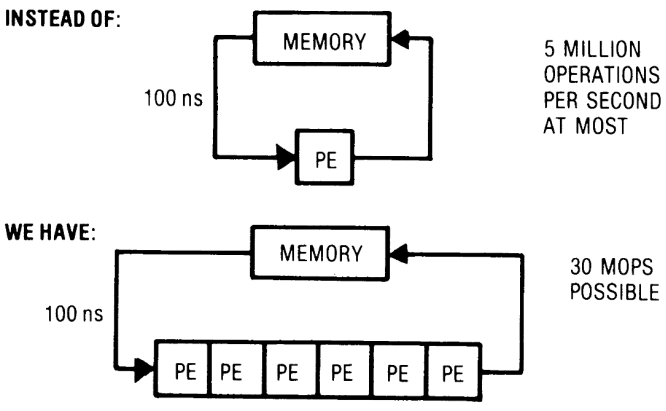
INT8 throughput	4 TOPS
Power	$\approx$ 2 W
Efficiency	2 TOPS/W
On-chip memory	8 MB
Off-chip memory	none, host only
Arithmetic	INT8 only
Workload	inference only
Host interface	USB 3.0

A systolic accelerator sized for a few watts, with no memory of its own beyond the 8 MB.

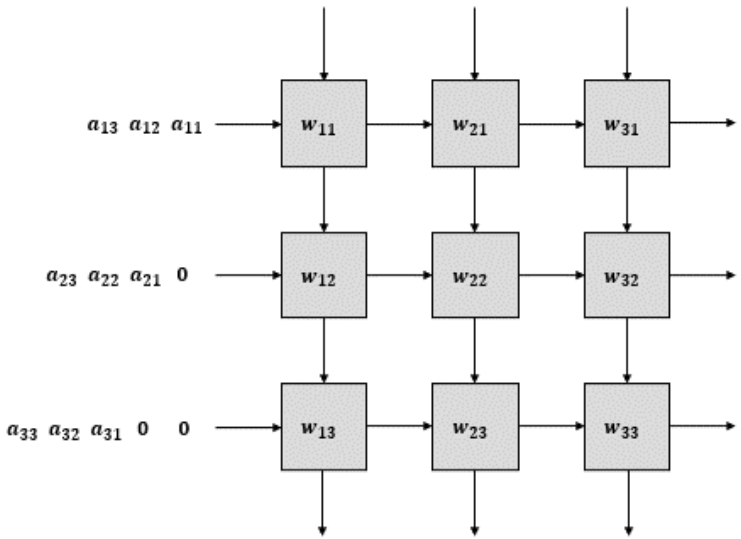
Edge TPU

Systolic array

$$\underbrace{\begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix}}_{\textbf{W weights, held in the PEs}} \times \underbrace{\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}}_{\textbf{A activations, streamed in}} = \underbrace{\begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix}}_{\textbf{C partial sums, flow down}}$$



H. T. Kung, *Why systolic architectures?* (1982)

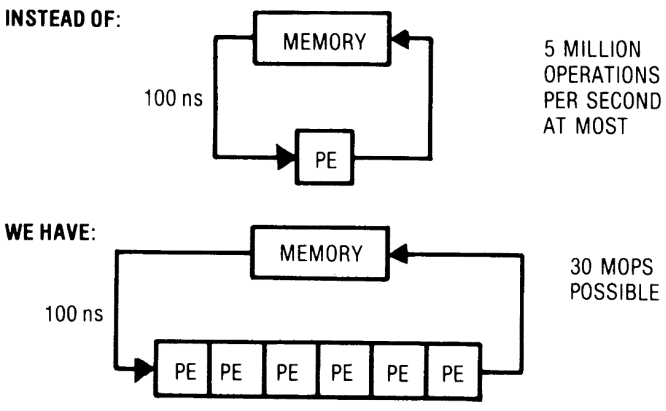


Weight-stationary dataflow: one operand fetch feeds a whole row.

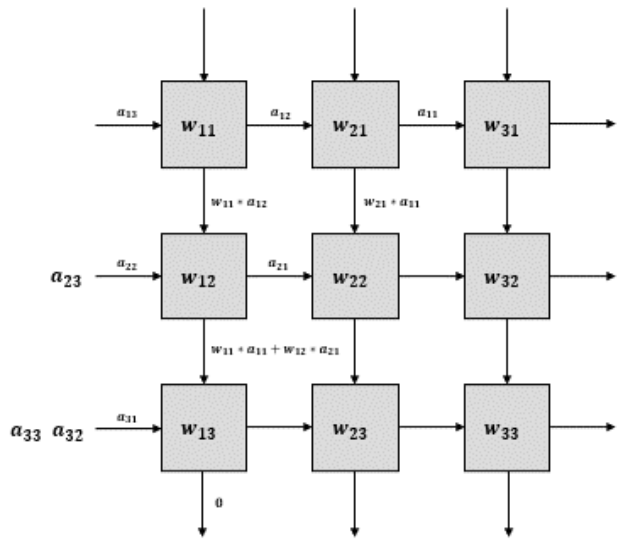
Edge TPU

Systolic array

$$\underbrace{\begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix}}_{\textbf{W weights, held in the PEs}} \times \underbrace{\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}}_{\textbf{A activations, streamed in}} = \underbrace{\begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix}}_{\textbf{C partial sums, flow down}}$$



H. T. Kung, *Why systolic architectures?* (1982)

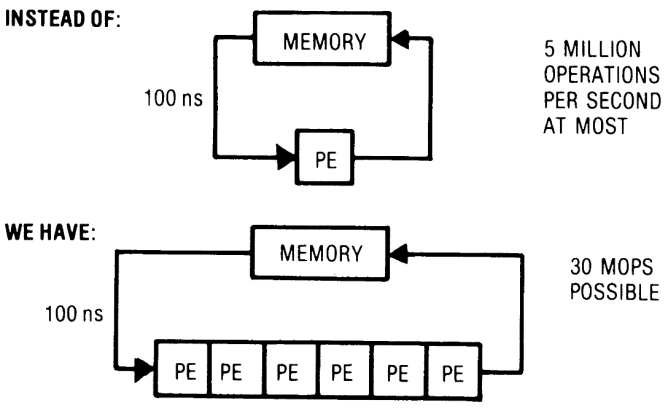


Weight-stationary dataflow: one operand fetch feeds a whole row.

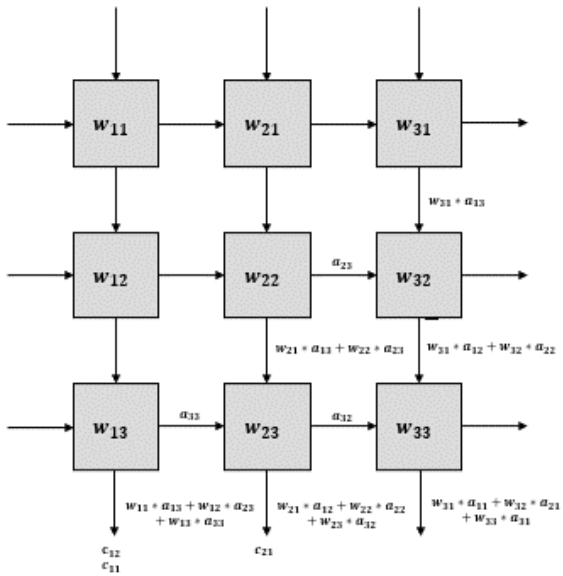
Edge TPU

Systolic array

$$\underbrace{\begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix}}_{\textbf{W} \text{ weights, held in the PEs}} \times \underbrace{\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}}_{\textbf{A} \text{ activations, streamed in}} = \underbrace{\begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix}}_{\textbf{C} \text{ partial sums, flow down}}$$



H. T. Kung, *Why systolic architectures?* (1982)

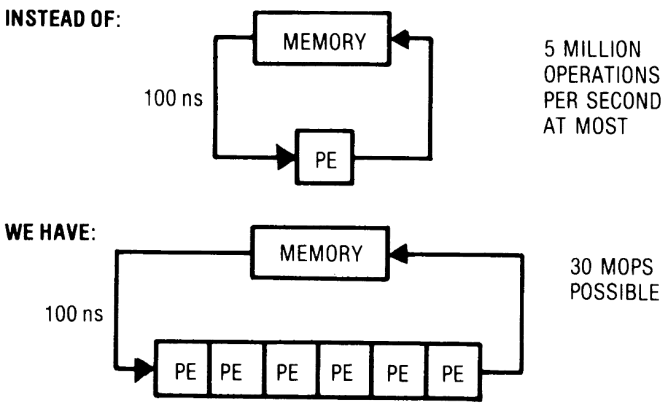


Weight-stationary dataflow: one operand fetch feeds a whole row.

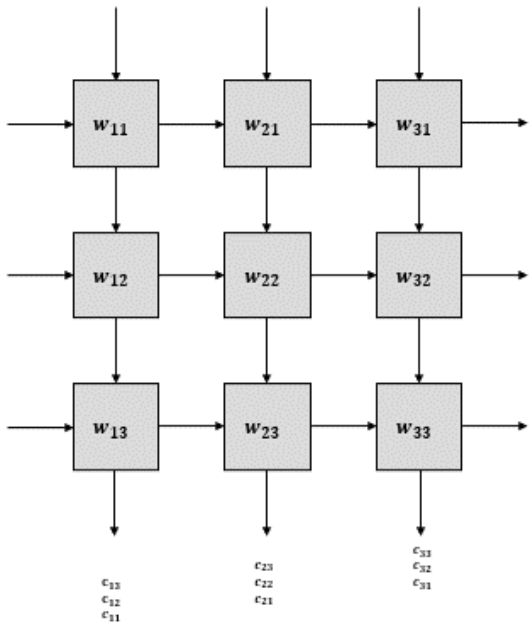
Edge TPU

Systolic array

$$\underbrace{\begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix}}_{\textbf{W weights, held in the PEs}} \times \underbrace{\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}}_{\textbf{A activations, streamed in}} = \underbrace{\begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix}}_{\textbf{C partial sums, flow down}}$$



H. T. Kung, *Why systolic architectures?* (1982)



Weight-stationary dataflow: one operand fetch feeds a whole row.

Edge TPU

Three constraints

- **A fixed set of operations**
  Conv, depthwise conv, pooling, add, concat, fully connected. The graph is cut at the first unsupported one.
- **INT8 only**
  Post-training quantization, 100 calibration images.
- **8 MB of on-chip memory**
  What does not fit is re-sent at *every* inference, ≈ 3.3 ms per MB.

The lighter the model, the lower its latency.

Performance of the Edge TPU

Steady-state latency, and model size split between on-chip SRAM and off-chip memory

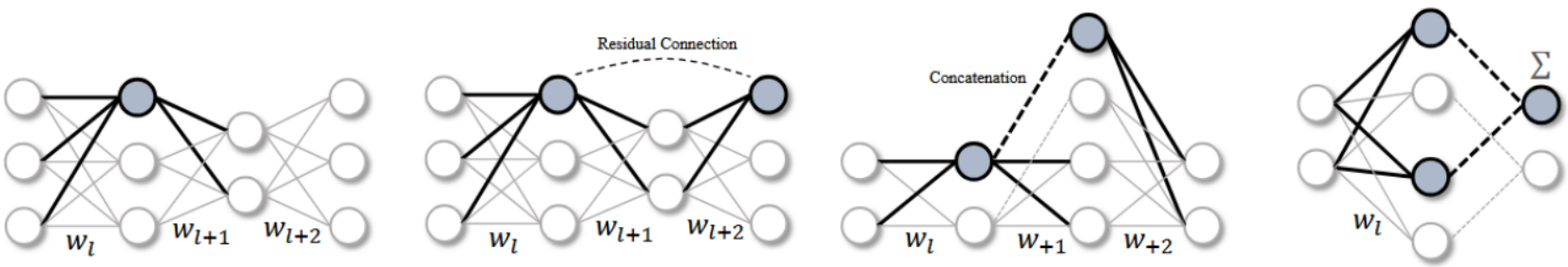
Model	CPU (ms)	TPU (ms)	On-chip (MB)	Off-chip (MB)	Speedup
GoogLeNet	63.0	2.0	5.66	**0.00**	**30.9×**
WRN-28-10	681.5	92.1	8.00	27.13	7.4×
ResNet-18	74.4	11.4	8.00	2.84	6.5×
MobileNetV2	9.7	1.2	2.70	0.00	7.8×
SqueezeNet 1.1	4.6	0.7	0.83	0.00	6.9×
ResNet-50	188.3	50.9	8.00	15.30	3.7×
VGG-19	70.7	39.3	8.00	**11.30**	**1.8×**

Cold start: first vs. second inference in a fresh interpreter

Model	1st inf. (ms)	2nd inf. (ms)	Ratio
GoogLeNet	21.43	2.13	**10.0×**
WRN-28-10	116.66	91.13	1.3×
ResNet-18	36.94	11.51	3.2×
MobileNetV2	10.97	1.34	8.2×
SqueezeNet 1.1	3.88	0.75	5.2×
ResNet-50	75.62	50.23	1.5×
VGG-19	63.92	38.79	1.7×

Structured pruning

- There are **two types of pruning**:
  - *Unstructured*: weights become zeros, tensor shapes do not change.
  - *Structured*: whole filters are removed, so the tensors really get smaller and the model really gets faster. **We choose this one.**
- The added difficulty is **dependency tracking**: cutting one filter changes the layers it feeds, and residual additions or concatenations tie several layers together. All of them must be cut consistently.
- Filters are ranked **across the whole network**: we set one target for the model, and the rate of each layer follows from the ranking.



(a) Basic dependency (b) Residual dependency (c) Concatenation dependency (d) Reduction dependency

Benchmark on the Coral USB Edge TPU

7 architectures, by topology	
Sequential	VGG-19
Residual	ResNet-18, ResNet-50, WRN-28-10
Inverted residual	MobileNetV2
Concatenation	GoogLeNet, SqueezeNet 1.1

7 criteria, by information used	
Data-free (weights)	Magnitude L1, L2, FPGM
Sparsity learning	BN-Scale
Data-driven (fwd+bwd)	Taylor, OBD-C
Control	Random

- **9 compression rates**
  10 % to 90 % of parameters removed, in steps of 10.

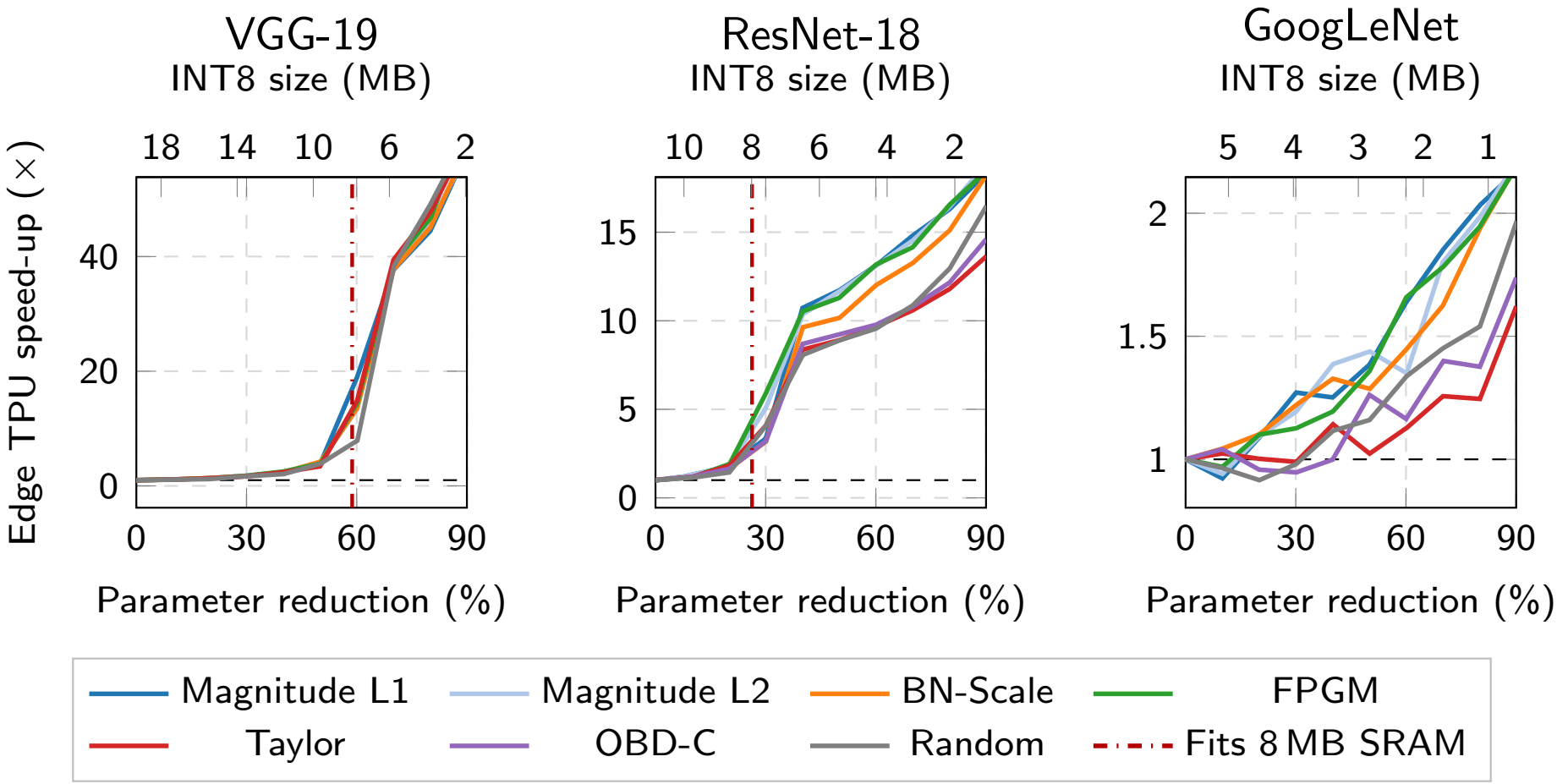
- **PruningBench protocol** [1]
  Unchanged: 400 iterative steps, global ranking, 100 recovery epochs.
  404 of the 441 combinations reached the accelerator.

CIFAR-100, 100 classes, 32×32

C. Li *et al.*, A Comprehensive Study of Structural Pruning for Vision Models (2025)

Latency of the pruned models

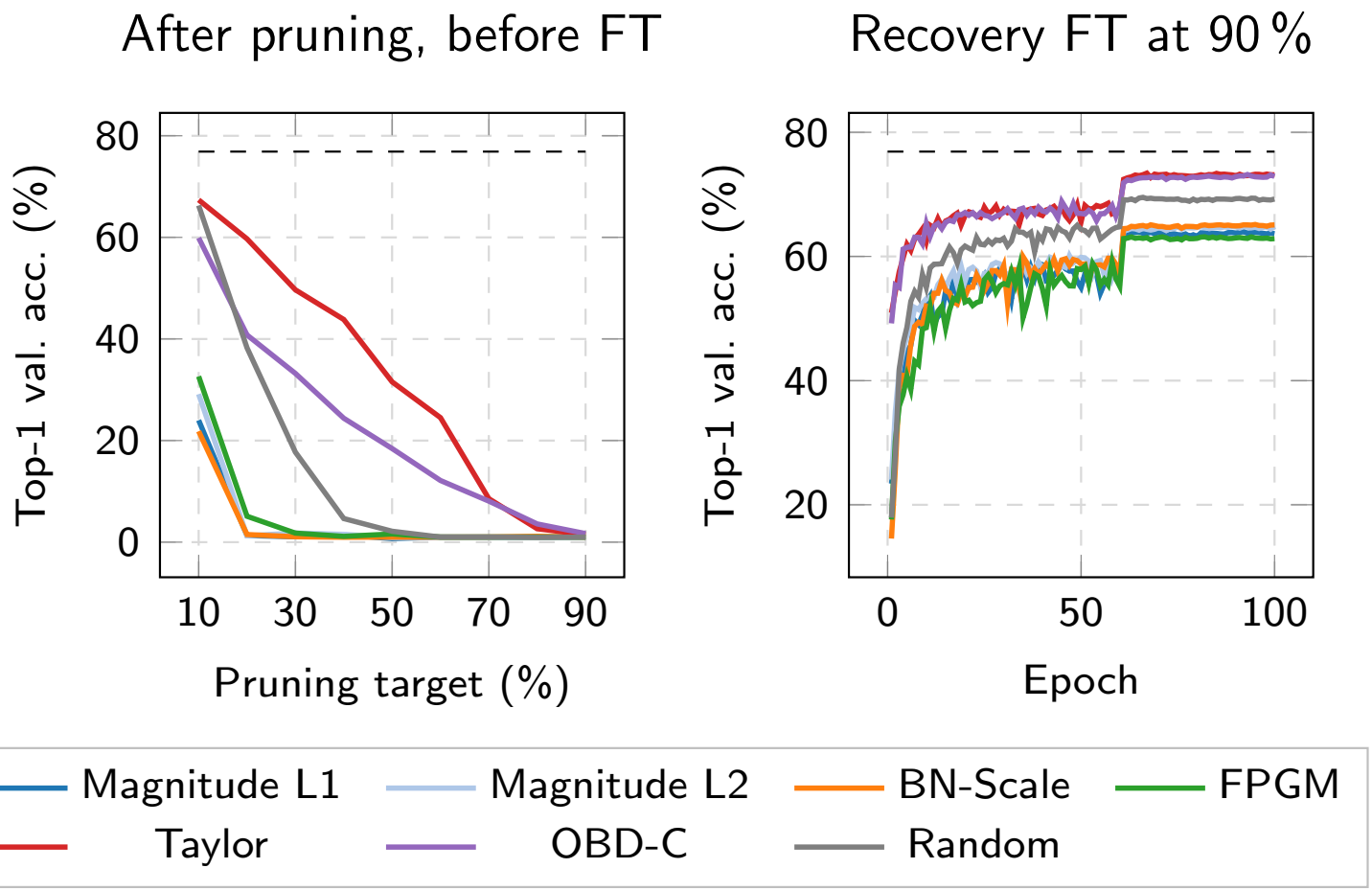
Edge TPU speed-up versus parameter reduction, and INT8 model size



Red line: the rate at which the model first fits entirely on chip. Note that each panel has its own vertical scale.

Accuracy

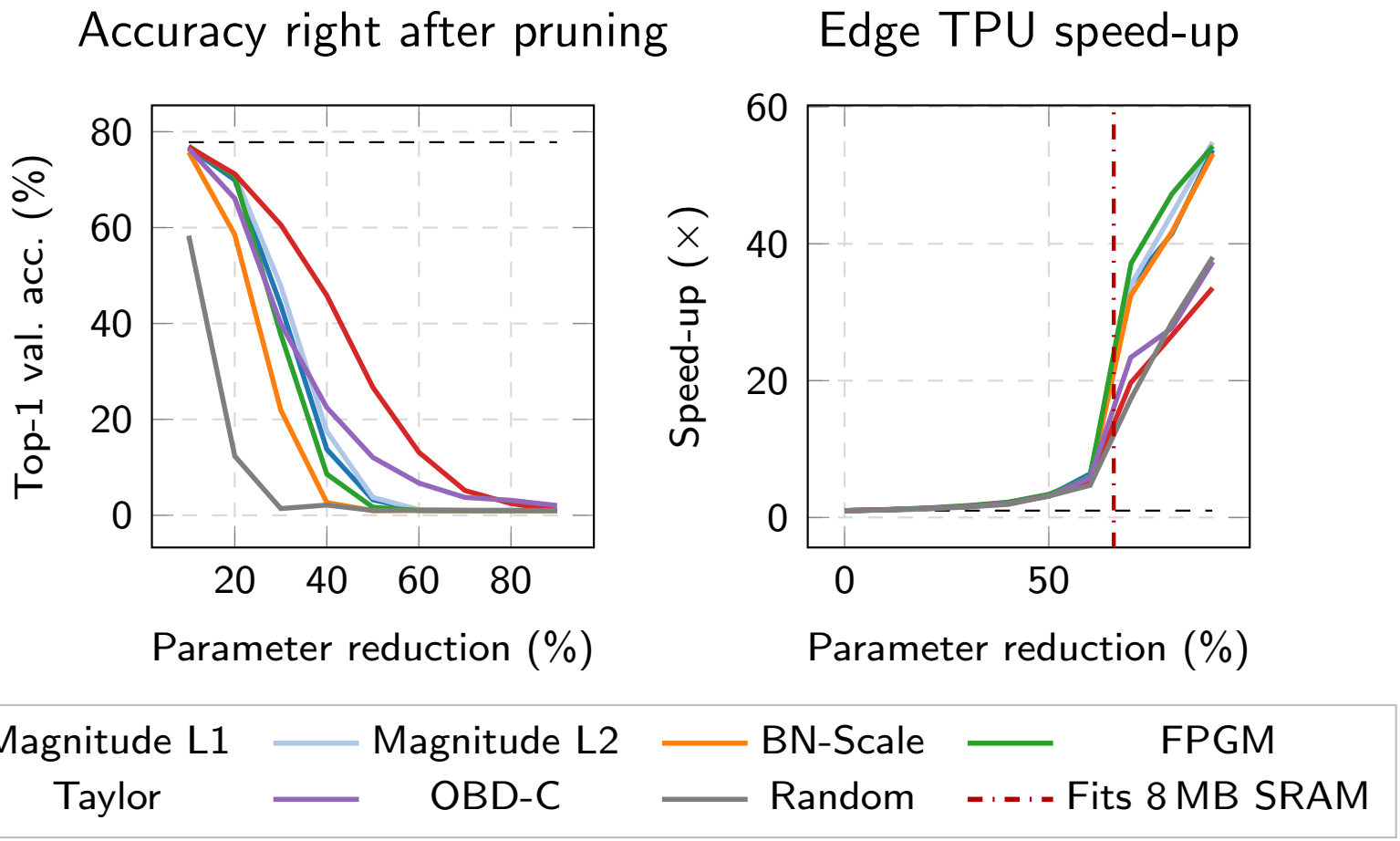
ResNet-18: accuracy right after pruning, and recovery fine-tuning



Dashed line: unpruned baseline. Both indicators give the same ranking.

Accuracy and latency disagree

ResNet-50: the criterion that best preserves accuracy is the slowest



At equal parameter reduction, two criteria do not produce the same on-chip footprint: *where* channels are removed matters.

Real-time scheduling of multiple DNNs on a single TPU

Illustrative use case

- Single TPU
- Many DNNs executed at different rates
- *Earliest Deadline First (EDF)*
- Non-preemptive avoids weight reloading
- Sporadic tasks (*i.e.*, event-triggered)

Function	Model	T (ms)	C (ms)	C/T
Nozzle trigger	ResNet-18	20	11.4	0.57
Crop growth stage	MobileNetV2	33	1.2	0.04
Weed species	ResNet-50	100	50.9	0.51
Ground cover	VGG-19	200	39.3	0.20
			Total utilization	**1.32**

$U > 1$: the task set is **infeasible**.

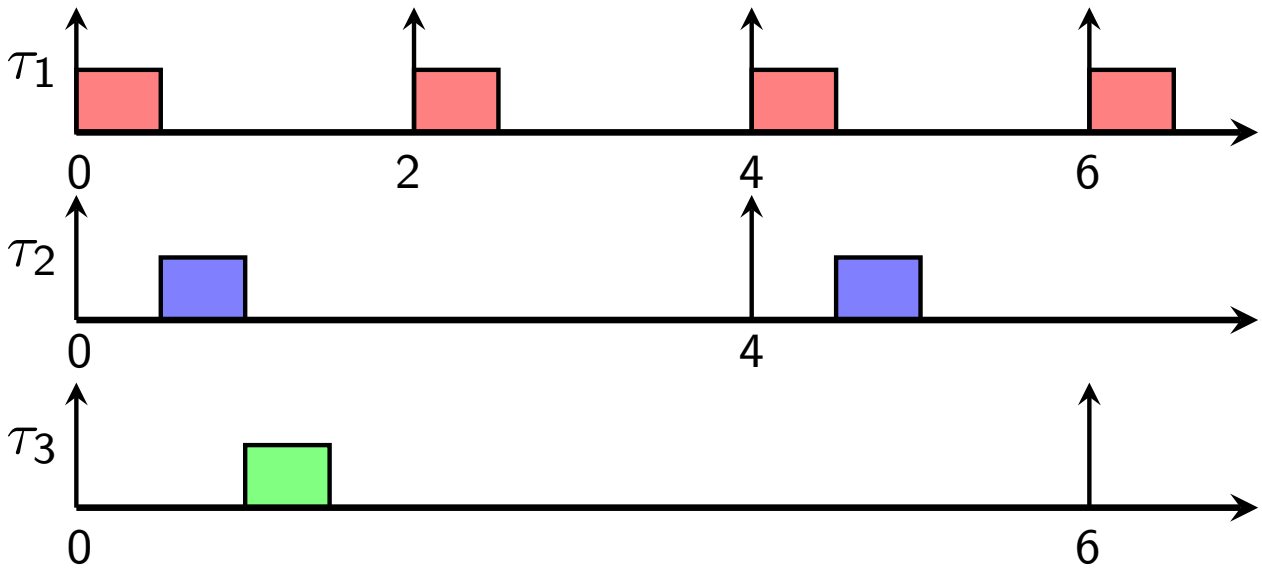
G. Buttazzo, Can We Trust AI-Powered Real-Time Embedded Systems? NG-RES (2022)

Periodic task scheduling

	C_i	T_i	A_i
τ_1	0.50	2.00	80.00%
τ_2	0.50	4.00	75.00%
τ_3	0.50	6.00	90.00%

Each sporadic task τ_i is characterized by:

- C_i worst-case inference time
- T_i minimum inter-arrival time (*period*)
- A_i accuracy



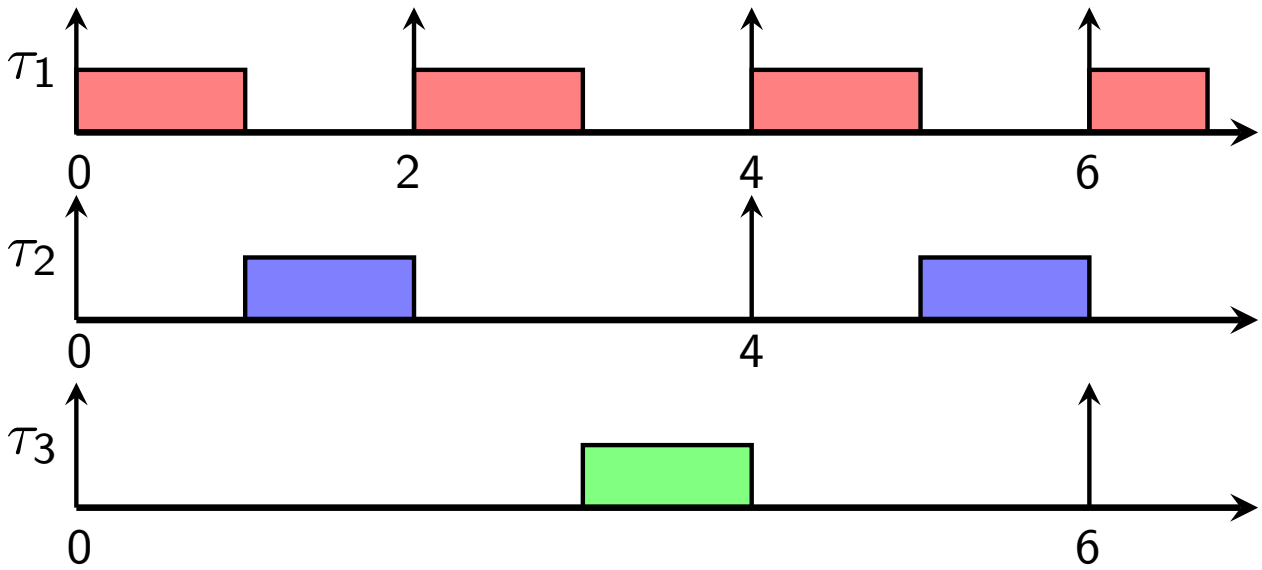
Schedulable but low accuracy

Periodic task scheduling

	C_i	T_i	A_i
τ_1	1.00	2.00	88.00%
τ_2	1.00	4.00	82.00%
τ_3	1.00	6.00	93.00%

Each sporadic task τ_i is characterized by:

- C_i worst-case inference time
- T_i minimum inter-arrival time (*period*)
- A_i accuracy



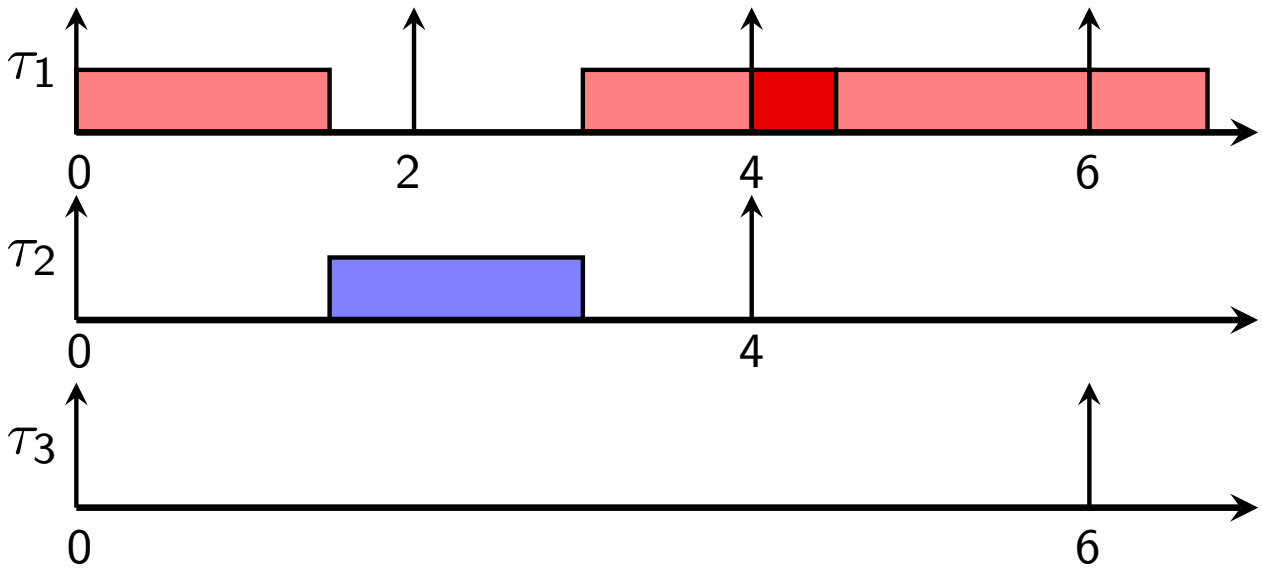
Schedulable and good accuracy

Periodic task scheduling

	C_i	T_i	A_i
τ_1	1.50	2.00	94.00%
τ_2	1.50	4.00	88.00%
τ_3	1.00	6.00	93.00%

Each sporadic task τ_i is characterized by:

- C_i worst-case inference time
- T_i minimum inter-arrival time (*period*)
- A_i accuracy



High accuracy but not schedulable

Non-preemptive EDF schedulability test

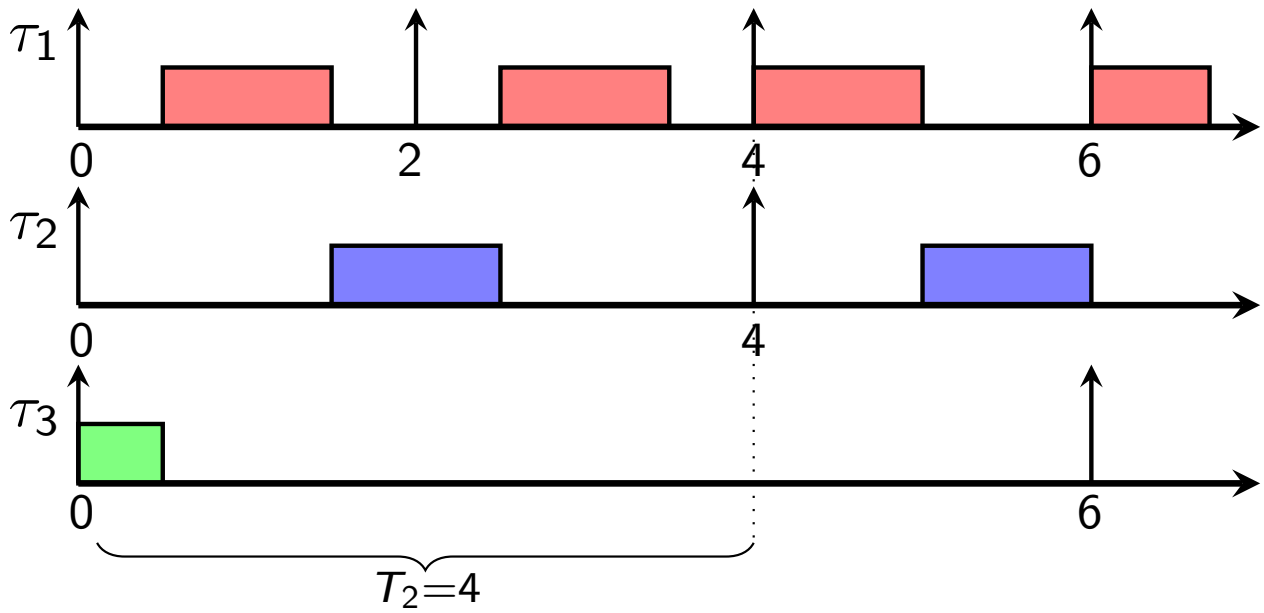
	C_i	T_i
τ_1	1.00	2.00
τ_2	1.00	4.00
τ_3	0.50	6.00

Sort tasks in non-decreasing order of periods.

For k from 1 to n:

$$\frac{B_k}{T_k} + \sum_{i=1}^{k} \frac{C_i}{T_i} \leq 1$$

where $B_k = \max\{C_j \mid T_j > T_k\}$



For k=2 : $B_2/T_2 + C_1/T_1 + C_2/T_2 = 0.5/4 + 1/2 + 1/4 = 0.875 \leq 1$

K. Jeffay, D.F. Stanat and C.U. Martel, On non-preemptive scheduling of period and sporadic tasks (1991)

0–1 Integer Linear Programming (ILP) model

$$\text{max.} \quad \sum_{i=1}^{n}\sum_{j=1}^{m} \frac{A_i^j}{T_i} \cdot x_i^j \qquad \text{Maximize accuracy} \quad (1)$$

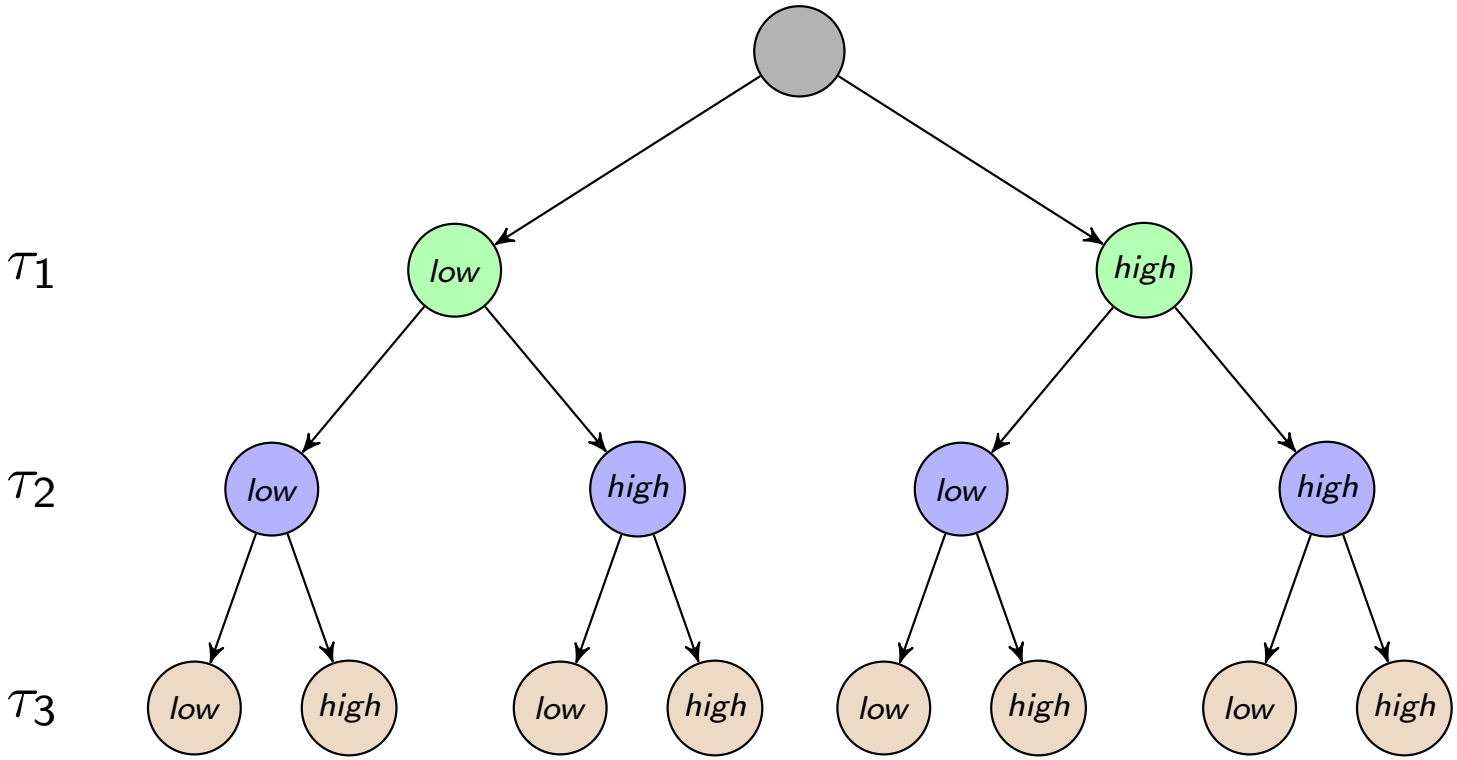
$$\text{s.t.} \quad B_k/T_k + \sum_{i=1}^{k}\sum_{j=1}^{m} x_i^j \cdot C_i^j/T_i \leq 1, \quad \forall\, 1 \leq k \leq n \qquad \text{Schedulability test} \quad (2)$$

$$B_k \geq C_i^j \cdot x_i^j, \quad \forall\, k < i \leq n, \ \forall\, 1 \leq j \leq m \qquad \text{Blocking factor} \quad (3)$$

$$\sum_{j=1}^{m} x_i^j = 1, \quad \forall\, 1 \leq i \leq n \qquad \text{One assignment} \quad (4)$$

$$x_i^j \in \{0,1\}, \quad \forall\, 1 \leq i \leq n, \ 1 \leq j \leq m \qquad \text{Binary decision variables} \quad (5)$$

Branch-and-bound (B&B)



Each level represents a task (τ_1, τ_2, τ_3).

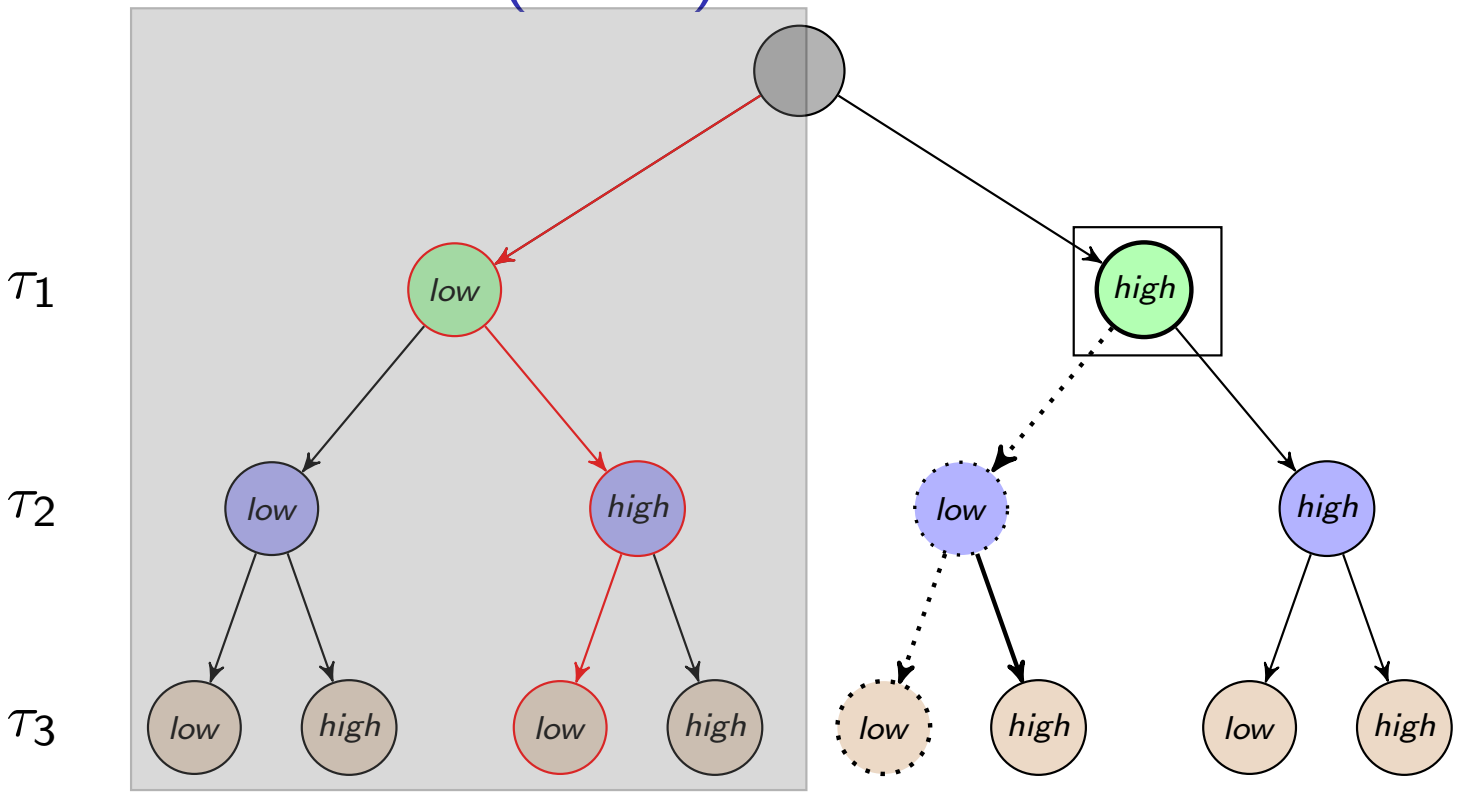
Each node represents an accuracy (*low* or *high*).

low accuracy $\Longrightarrow$ low inference times while high accuracy $\Longrightarrow$ high inference times.

Find a pruning rate assignment for each task such that:

i) the average precision is maximized, and ii) tasks are schedulable.

Branch-and-bound (B&B)



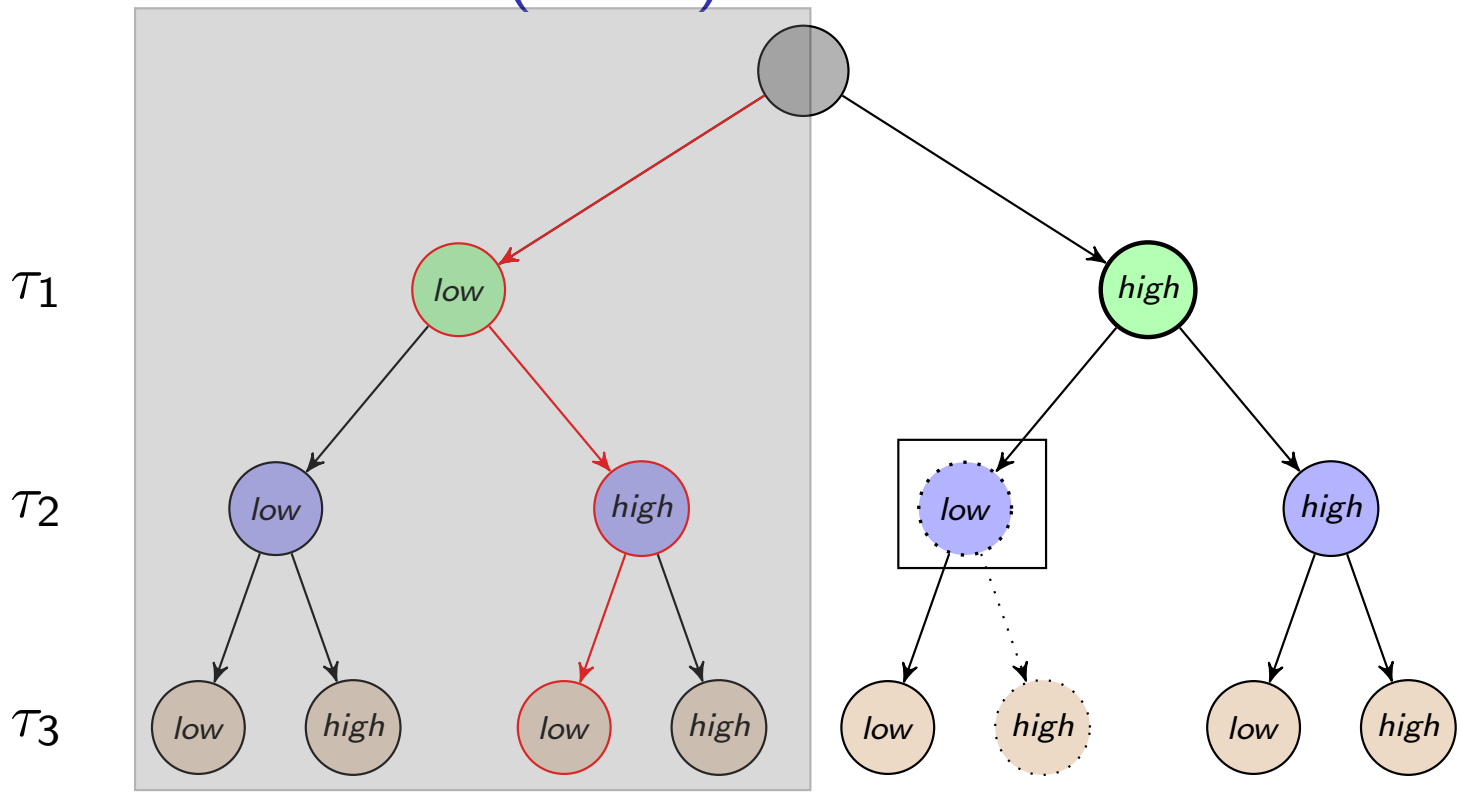
Current state

Left sub-tree has been explored, we are at $\tau_1 \leftarrow \mathit{high}$, and $\tau_1 \leftarrow \mathit{low}, \tau_2 \leftarrow \mathit{high}, \tau_3 \leftarrow \mathit{low}$ is the best solution found so far.

Schedulability branch-pruning

If $\tau_1 \leftarrow \mathit{high}, \tau_2 \leftarrow \mathbf{low}, \tau_3 \leftarrow \mathbf{low}$ is not schedulable then no other assignment can be schedulable.

Branch-and-bound (B&B)



Current state

Left sub-tree has been explored, we are at $\tau_1 \leftarrow \mathit{high}, \tau_2 \leftarrow \mathit{low}$, and $\tau_1 \leftarrow \mathit{low}, \tau_2 \leftarrow \mathit{high}, \tau_3 \leftarrow \mathit{low}$ is still the best solution found so far.

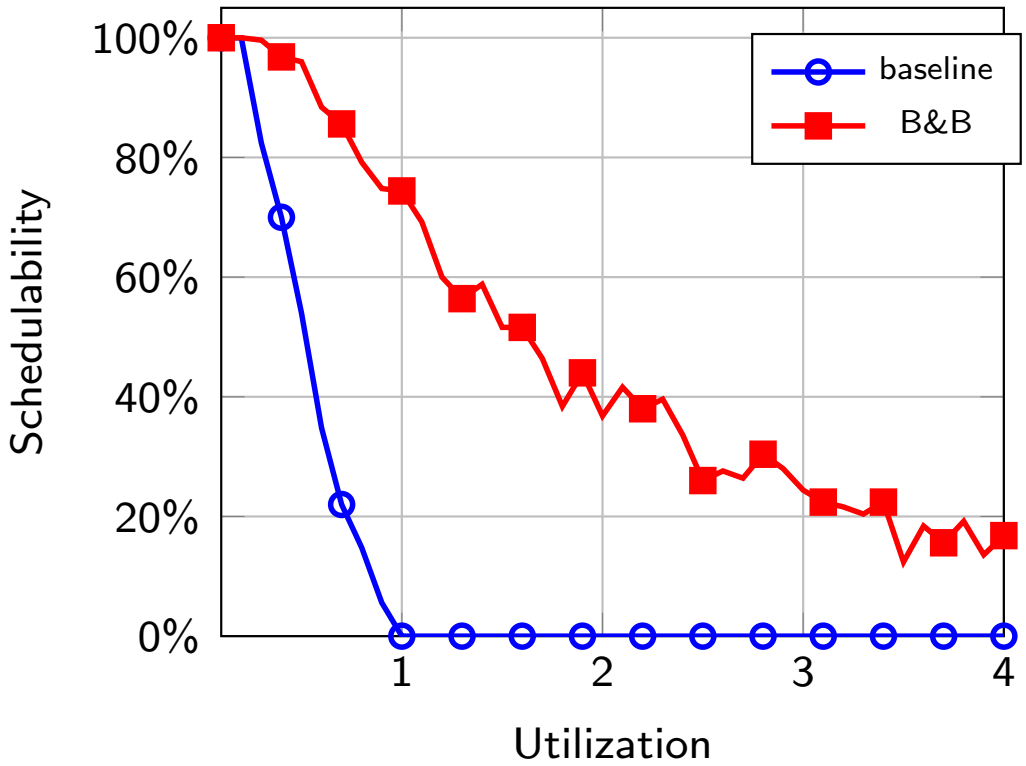
Accuracy branch-pruning

If $\tau_1 \leftarrow \mathit{high}, \tau_2 \leftarrow \mathit{low}, \tau_3 \leftarrow$ **high** does not have better accuracy then no other assignment can be better (*i.e.*, $\tau_3 \leftarrow \mathit{low}$).

Schedulability experiments

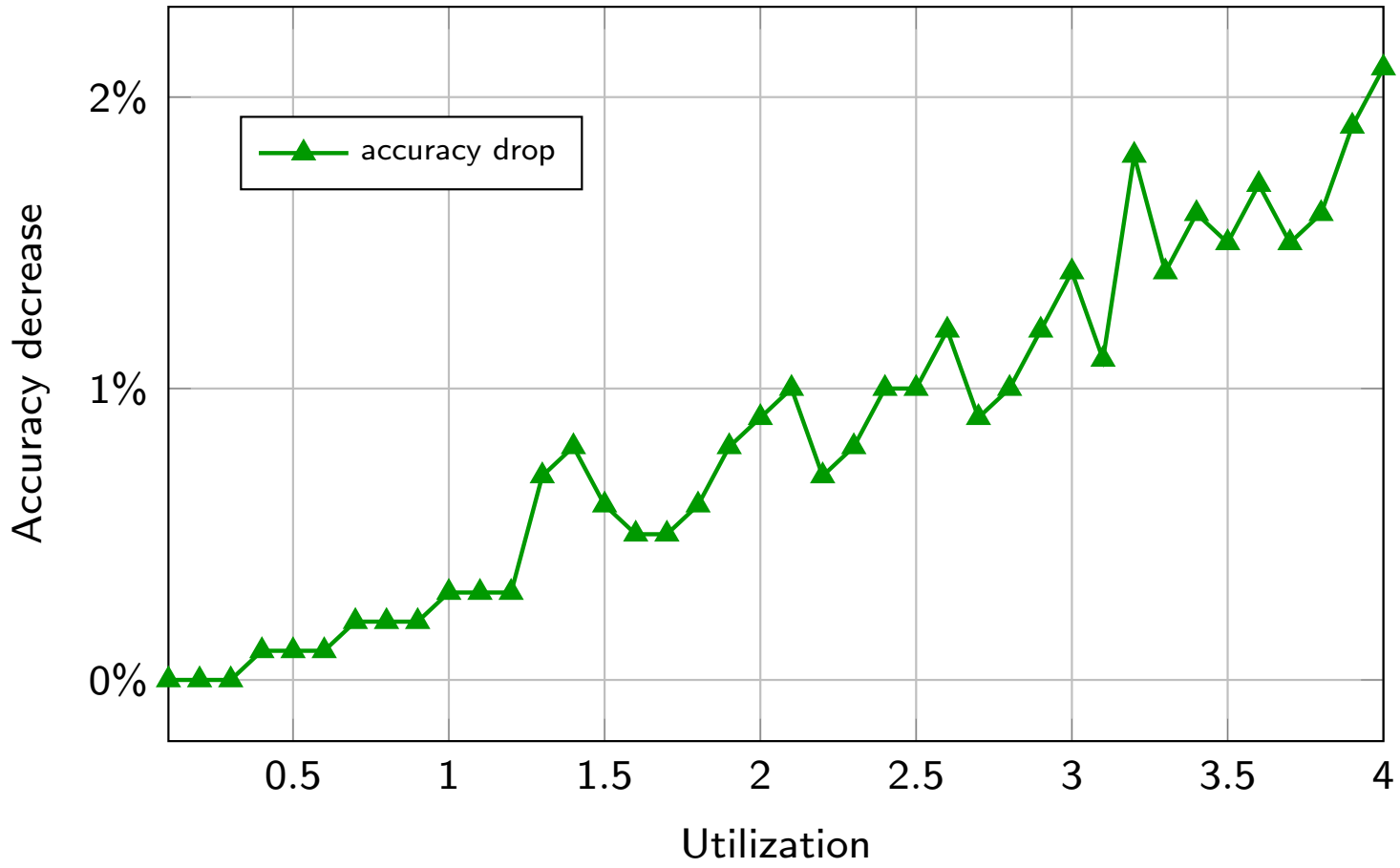
Schedulability ratio with and without pruning

- Random task sets of **six tasks**, utilizations drawn with **RandFixedSum**.
- **250 task sets** per utilization point, from 0.1 to 4.0.
- Periods sampled **log-uniformly** between 10 and 100 ms.
- Each task takes its **profile at random** from the benchmark.



Schedulability experiments

Accuracy drop with pruning for B&B method



Conclusion

- **Latency is a step function of model size.**
  Up to 60× below the 8 MB threshold, at most 2× above it. The knee is the on-chip capacity, not the parameter count.
- **No importance criterion dominates both dimensions.**
  Criterion selection is itself an accuracy–latency trade-off.
- **Pruning rates are a scheduling variable.**
  Schedulable well beyond the point where the unpruned system fails, for about 2 % of accuracy on the most loaded task sets.

Thank you for your attention.